

P3016R6

Resolve inconsistencies in `begin/end` for `valarray` and braced initializer lists

Published Proposal, 2025-02-11

Author:

[Arthur O'Dwyer](#)

Audience:

LWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

We resolve some inconsistencies among [iterator.range] functions, notably `begin/end`, `data`, and `empty`, as applied to `valarray`, `initializer_list`, and braced initializer lists; and notably harmonizing their noexcept-specs. This resolves LWG3537, LWG3624, LWG3625, and LWG4131.

Table of Contents

- 1 Changelog**
- 2 Motivation and proposal**
 - 2.1 `data` and `empty`
 - 2.2 Availability of `begin/end`
 - 2.3 Noexcept-specs
- 3 Implementation experience**
 - 3.1 For the new noexcept-specs
 - 3.2 For the `initializer_list` and `valarray` changes
 - 3.3 Tony Table
- 4 Proposed wording**
 - 4.1 Resolve LWG3624
 - 4.2 [version.syn]
 - 4.3 [valarray.syn]
 - 4.4 [template.valarray.overview]
 - 4.5 [valarray.members]
 - 4.6 [support.initlist]
 - 4.7 [iterator.synopsis]

- 4.8 [iterator.range]
- 4.9 .data+.size cleanup
- 4.9.1 [string.cons]
- 4.9.2 [string.append]
- 4.9.3 [string.assign]
- 4.9.4 [string.replace]
- 4.9.5 [span.cons]
- 4.9.6 [valarray.cons]

References

Informative References

§ 1. Changelog

- R6 (pre-LWG in Hagenberg):
 - LEWG voted 16–11–0–1–0 to forward [\[P3623R0\]](#) to LWG, adding noexcept-specs to all ten generic free functions. P3016R6 merges that wording so that LWG can see only this one combined paper. This additionally resolves [\[LWG3537\]](#).
- R5 (during Wrocław):
 - LWG voted 7–1–0 to forward R4 to plenary, but then retracted that vote and asked LEWG to re-confirm the design decision that the generic free functions [begin](#), [data](#), etc. remain non-noexcept. LWG asks LEWG to discuss and poll that question. Then this paper can return to LWG.
 - Add [<typeindex>](#) to the [resolution for LWG3624](#).
 - Merge LWG issue resolutions back into "Proposed Wording," per jwakely; LWG approves of these resolutions.
- R4 (pre-LWG in Wrocław):
 - Add [resolution for LWG4131](#).
- R3 (post-LEWG in Tokyo):
 - Move possible resolutions for LWG3624 and LWG3625 out of the "Proposed Wording" section; LEWG didn't want to step on LWG's toes by seeming to bless these issue resolutions
 - For the same reason, remove [#include <initializer_list>](#) from the proposed wording of [iterator.synopsis]; it remains in the possible resolutions for LWG3624 and LWG3625.
- R2:
 - Give resolutions for LWG3624 and LWG3625
 - Rebut concerns about [breaking code](#); discuss how a deprecation period for [std::begin\(initializer_list\)](#) is basically unimplementable
 - Make [valarray::iterator](#) non-exposition-only
 - Remove R1's [Historical Background](#) section, as basically distracting at this point
 - Mention [\[P2613\]](#) and [\[LWG4035\]](#) as further precedent for [initializer_list.empty\(\)](#)
 - Add feature-test macros [__cpp_lib_initializer_list](#) and [__cpp_lib_valarray](#)
- R1:

- Split [\[CWG2825\]](#) from this paper; this paper is now library-only
- Add discussion of `.data()` and `.empty()` overloads
- Add wording for ".data+.size cleanup"

§ 2. Motivation and proposal

Casey Carter [points out](#) that the following program is supported by libstdc++ but not libc++ nor Microsoft ([Godbolt](#)):

```
#include <iterator>
#include <valarray>
int main() {
    std::valarray<int> v = {1,2,3};
    std::begin(v); // OK
    std::cbegin(v); // Error
}
```

This is because `std::valarray` defines its own non-member, non-hidden-friend overloads of `std::begin` and `std::end`. These overloads are found by the qualified call to `std::begin` here, but aren't found by `std::cbegin`'s ADL because the primary template for `std::cbegin` happens to be defined before `<valarray>` is included. Swapping the order of `#include <iterator>` and `#include <valarray>` in this example doesn't help, because the relevant parts of `<iterator>` are still transitively included by `<valarray>` before `std::valarray`'s own code.

Likewise, on all vendors ([Godbolt](#)):

```
#include <iterator>
int main() {
    std::begin({1,2,3}); // OK
    std::cbegin({1,2,3}); // Error
}
```

This is because `{1,2,3}` is a *braced-initializer-list* with no type; so it cannot bind to the deduced `const C&` in `std::cbegin` (defined in `<iterator>`). But it *can* bind to the `initializer_list<E>` in the non-member, non-hidden-friend overload `std::begin(std::initializer_list<E>)` (defined in `<initializer_list>`).

Notice that `std::begin({1,2,3})` returns an iterator that will dangle at the end of the full-expression, and that the return values of `std::begin({1,2,3})` and `std::end({1,2,3})` do not form a range (because the two lists' backing arrays may be different). Therefore this overload is more harmful than helpful.

NOTE: Be careful to distinguish the scenario of calling `std::begin` on an *object of type* `initializer_list` (helpful!) from calling it on a *braced-initializer-list* (harmful).

We propose to resolve `valarray`'s `begin/cbegin` inconsistency in favor of "make it work" and to resolve *braced-initializer-list*'s `begin/cbegin` inconsistency in favor of "make it ill-formed."

§ 2.1. `data` and `empty`

We also propose two more member functions for `std::initializer_list`: `.data()` and `.empty()`.

Many places in the library clauses would like to operate on the contiguous data of an `initializer_list` using the "data + size" idiom, but since `initializer_list` lacks `.data()`, they're forced to use an awkward "begin + size" approach instead. This (1) looks unnatural and (2) causes extra mental effort for library writers. Part of P3016's proposed wording is to update these places in the library. For example:

```
constexpr basic_string& append(const basic_string& str);

1. Effects: Equivalent to return append(str.data(), str.size());

[...]

constexpr basic_string& append(initializer_list<charT> il);

16. Effects: Equivalent to return append(il.data(), il.size() il.begin(), il.size());
```

As for `.empty()`, it is generally recognized these days that ranges providing `.size()` should also provide `.empty()`. For example, `iota_view`'s missing `.empty()` was added as a DR by [LWG4001]. [LWG4035] adds `single_view`'s missing `.empty()`. [P2613], which added `mdspan`'s missing `.empty()`, claims that `initializer_list` and `bitset` remain the only two types with `.size()` and without `.empty()`.

By making `il.data()` and `il.empty()` well-formed for `std::initializer_list` objects, we satisfy the SFINAE conditions of the primary templates for `std::data(il)` and `std::empty(il)`, meaning that we can eliminate their special overloads for `initializer_list` arguments.

<pre>std::initializer_list<int> il = {1,2,3}; const int *p = std::data(il); // OK, calls the initializer_list overload // specially provided in <iterator> bool b = std::empty(il); // OK, calls the initializer_list overload // specially provided in <iterator></pre>	<pre>std::initializer_list<int> il = {1,2,3}; const int *p = std::data(il); // OK, calls the primary template bool b = std::empty(il); // OK, calls the primary template</pre>
<pre>const int *p = std::data({1,2,3}); // calls the initializer_list overload // specially provided in <iterator>, // and immediately dangles const int *q = std::data({}); // does not compile</pre>	<pre>const int *p = std::data({1,2,3}); // does not compile const int *q = std::data({}); // does not compile</pre>
<pre>bool b = std::empty({1,2,3}); // calls the initializer_list overload // specially provided in <iterator>, // invariably returns false bool c = std::empty({}); // does not compile</pre>	<pre>bool b = std::empty({1,2,3}); // does not compile bool c = std::empty({}); // does not compile</pre>

§ 2.2. Availability of `begin/end`

The following snippet tries to use `std::begin(il)` without first including `<iterator>` nor any of the 15 other headers listed in [\[iterator.range\]/1](#) that declare the generic `std::begin`. This code works in C++23.

We propose simply to break the left-hand snippet, and force the programmer to write the right-hand snippet instead. One LEWG reviewer expressed concern that we were breaking valid code without a deprecation period.

```
#include <initializer_list>
std::initializer_list<int> il = {1,2,3};
const int *b = std::begin(il);
    // OK, calls the initializer_list overload
    // specially provided in <initializer_list>
const auto r = std::rbegin(il);
    // Error, not declared in <initializer_list>
const int *c = std::cbegin(il);
    // Error, not declared in <initializer_list>
size_t s = std::size(il);
    // Error, not declared in <initializer_list>
```

```
#include <iterator>
std::initializer_list<int> il = {1,2,3};
const int *b = std::begin(il);
    // OK, calls the primary template
const auto r = std::rbegin(il);
    // OK, calls the initializer_list overload
    // specially provided in <iterator>
const int *c = std::cbegin(il);
    // OK, calls the primary template
size_t s = std::size(il);
    // OK, calls the primary template
```

At first, I was amenable to the idea that the declarations of `std::begin` and `std::end` in `<initializer_list>` should merely be deprecated, not fully removed in C++26, so that the behavior of the code on the left would be (temporarily) preserved. To do that, we'd modify [\[iterator.range\]](#) like this:

```
template<class C> constexpr auto begin(C& c) -> decltype(c.begin());
template<class C> constexpr auto begin(const C& c) -> decltype(c.begin());
```

2. Returns: `c.begin()`.

x. Remarks: In addition to being available via inclusion of the `<iterator>` header, these function templates are available when `<initializer_list>` is included. This availability is deprecated.

But library vendors can't implement that wording! We have `[[deprecated]]` to mark an *entity* as deprecated, but we have no way to mark a *single declaration* as deprecated. (I.e., "If, hypothetically, removing this declaration would make the call ill-formed, then give a deprecation warning; otherwise don't.") Three years from now, we'd come back asking "Can we remove this deprecated feature yet?" and the answer would be "No, vendors haven't started giving a warning for it yet," because it's physically impossible to give a warning for it. This would be a silly situation to get into. And the feature itself is so obscure (using `std::begin` specifically on an `initializer_list` without including any of more than a dozen STL headers) and so specific (as shown, it already doesn't work for `cbegin`, `rbegin`, or `size`), and the fix in code is so surgical (include `<iterator>` or `<span>`) that I don't think the lack of deprecation period will matter.

§ 2.3. Noexcept-specs

In the paper standard for C++23, we find ([\[iterator.range\]](#), [\[valarray.syn\]](#), [\[initializer.list.syn\]](#), [\[fs.filesystem.syn\]](#)):

- `begin`, `end`: noexcept for arrays, `initializer_list`, and `[recursive_]directory_iterator`; otherwise never-noexcept
- `data`, `empty`: noexcept for arrays and `initializer_list`; otherwise never-noexcept
- `cbegin`, `cend`: conditionally noexcept

- `rbegin`, `rend`, `crbegin`, `crend`: never-noexcept
- `size`, `ssize`: noexcept for arrays; otherwise never-noexcept

This is explained by two historical quirks:

- The generic `cbegin` and `cend` are conditionally noexcept, despite no other generic algorithm being so. (This is because there are no dedicated overloads of `cbegin` or `cend`, because [LWG2128] didn't think we needed them. Then, when [LWG2280] wanted to make the array and `initializer_list` overloads `noexcept`, there were no dedicated overloads to modify, so we just added a conditional noexcept-spec to the generic template.)
- The dedicated overloads of `rbegin` and `rend` are never-noexcept, despite every other dedicated overload being so. (I suspect this is because `reverse_iterator`'s constructor remains never-noexcept. [N3263] guaranteed that `ctr.rbegin()` would be noexcept for all STL containers, but made no such guarantee about reversing an *arbitrary* iterator.)

We propose to add `cbegin`-style conditional noexcept-specs everywhere we can.

§ 3. Implementation experience

§ 3.1. For the new noexcept-specs

Yes: fully on Microsoft, "mostly" on libstdc++, "partly" on libc++.

Microsoft STL has put noexcept-specs on `cbegin`, `cend` since pre-2019; on `begin`, `end`, `rbegin`, `rend`, `crbegin`, `crend`, `data`, `size`, `ssize`, `empty` since [August 2022](#).

GNU libstdc++ has put noexcept-specs on `cbegin`, `cend` since [January 2015](#); on `data`, `size`, `empty` since [November 2017](#); on `begin`, `end`, `ssize` since [November 2024](#). It continues to leave unmarked the generic `rbegin` and `rend`.

libc++ has put noexcept-specs on `data`, `size`, `empty` since [November 2017](#); on `ssize` since [February 2019](#); on `cbegin`, `cend` since [October 2023](#). It continues to leave unmarked the generic `begin`, `end`, `rbegin`, `rend`, `crbegin`, and `crend`.

§ 3.2. For the `initializer_list` and `valarray` changes

Arthur has implemented [§ 4 Proposed wording](#) in his fork of libc++ ([source](#)), and used it to compile both LLVM/Clang/libc++ and another large C++17 codebase. Naturally, it caused no problems except in this single test from libc++'s own test suite:

```
#include <initializer_list> // but not <iterator>
std::initializer_list<int> il;
static_assert(noexcept(std::begin(il)));
```

This test now fails because `<iterator>` was not included.

§ 3.3. Tony Table

<pre>#include <initializer_list> void f(std::initializer_list<int> il) { auto it = std::begin(il); }</pre>	<pre>#include <initializer_list> #include <iterator> // for std::begin void f(std::initializer_list<int> il) { auto it = std::begin(il); }</pre>
<pre>S(std::initializer_list<int> il) : S(il.begin(), il.size()) {}</pre>	<pre>S(std::initializer_list<int> il) : S(il.data(), il.size()) {}</pre>
<pre>auto dangle = std::begin({1,2,3}); // calls the initializer_list overload // specially provided in <initializer_list>, // and immediately dangles</pre>	<pre>// no longer compiles</pre>
<pre>bool b = std::empty({1,2,3}); // calls the initializer_list overload // specially provided in <iterator>, // yielding false</pre>	<pre>// no longer compiles</pre>
<pre>#include <valarray> #include <utility> std::valarray<int> va; auto it = std::begin(std::as_const(va));</pre>	<pre>#include <valarray> #include <iterator> std::valarray<int> va; auto it = std::cbegin(va);</pre>
<pre>std::vector<int> v; int arr[10]; static_assert(noexcept(v.begin())); // guaranteed static_assert(noexcept(std::begin(arr))); // guaranteed static_assert(noexcept(std::begin(v))); // passes on MS and libstdc++, fails on libc++ static_assert(noexcept(v.rbegin())); // guaranteed static_assert(noexcept(std::rbegin(arr))); // passes on MS and libstdc++, fails on libc++ static_assert(noexcept(std::rbegin(v))); // passes on MS, fails on libstdc++ and libc++</pre>	<pre>std::vector<int> v; int arr[10]; static_assert(noexcept(v.begin())); // guaranteed static_assert(noexcept(std::begin(arr))); // guaranteed static_assert(noexcept(std::begin(v))); // guaranteed static_assert(noexcept(v.rbegin())); // guaranteed static_assert(noexcept(std::rbegin(arr))); // guaranteed static_assert(noexcept(std::rbegin(v))); // guaranteed</pre>

§ 4. Proposed wording

NOTE: Vendors should provide `__cpp_lib_initializer_list` if `il.data()` and `il.empty()` are well-formed. Vendors should provide `__cpp_lib_valarray` if `va.begin()` and `va.end()` are well-formed and the free-function `begin` and `end` have been removed from `<valarray>`.

§ 4.1. Resolve LWG3624

The following changes resolve [LWG3624].

Modify [iterator.synopsis] as follows:

```
#include <compare>           // see [compare.syn]
#include <concepts>          // see [concepts.syn]
#include <initializer_list>   // see [initializer.list.syn]
```

Modify [any.synop] as follows (since [any.class] requires both `std::initializer_list` and `std::type_info`):

```
#include <initializer_list>   // see [initializer.list.syn]
#include <typeinfo>           // see [typeinfo.syn]
```

```
namespace std {
```

Modify [functional.syn] as follows (since [func.wrap.move] requires `std::initializer_list` and [func.wrap.func] requires `std::type_info`):

```
#include <initializer_list>   // see [initializer.list.syn]
#include <typeinfo>           // see [typeinfo.syn]
```

```
namespace std {
```

Modify [type.index.synopsis] as follows (since [type.index.overview] requires `std::type_info`):

```
#include <compare>           // see [compare.syn]
#include <typeinfo>           // see [typeinfo.syn]
```

```
namespace std {
```

Contra LWG3624, do not modify [stacktrace.syn]. It uses `std::initializer_list` indirectly, via [iterator.range]/1 — but it uses `std::reverse_iterator` indirectly in the same way, and we certainly don't expect it to include `<iterator>`! The exact mechanism by which the library vendor satisfies [dcl.init.list]/2 should be left unspecified in that case.

§ 4.2. [version.syn]

Add two feature-test macros to [version.syn]/2:


```

#define __cpp_lib_incomplete_container_elements 201505L // also in <forward_list>, <list>, <vector>
#define __cpp_lib_initializer_list YYYMMML // also in <initializer_list>
#define __cpp_lib_int_pow2 202002L // freestanding, also in <bit>
[...]
```

```

#define __cpp_lib_unwrap_ref 201811L // freestanding, also in <type_traits>
#define __cpp_lib_valarray YYYMMML // also in <valarray>
#define __cpp_lib_variant 202306L // also in <variant>
```

§ 4.3. [valarray.syn]

Modify [valarray.syn] as follows:

[...]

```

template<class T> valarray<T> tan (const valarray<T>&);
template<class T> valarray<T> tanh (const valarray<T>&);

template<class T> unspecified1 begin(valarray<T>& v);
template<class T> unspecified2 begin(const valarray<T>& v);
template<class T> unspecified1 end(valarray<T>& v);
template<class T> unspecified2 end(const valarray<T>& v);
}
```

[...]

3. Any function returning a `valarray<T>` is permitted to return an object of another type, provided all the const member functions of `valarray<T>` **other than `begin` and `end`** are also applicable to this type. This return type shall not add more than two levels of template nesting over the most deeply nested argument type.

4. Implementations introducing such replacement types shall provide additional functions and operators as follows:

- (4.1) for every function taking a const `valarray<T>&` **other than `begin` and `end`**, identical functions taking the replacement types shall be added;
- (4.2) for every function taking two const `valarray<T>&` arguments, identical functions taking every combination of const `valarray<T>&` and replacement types shall be added.

5. In particular, an implementation shall allow a `valarray<T>` to be constructed from such replacement types and shall allow assignments and compound assignments of such types to `valarray<T>`, `slice_array<T>`, `gslice_array<T>`, `mask_array<T>` and `indirect_array<T>` objects.

[...]

§ 4.4. [template.valarray.overview]

NOTE: R1 proposed the `iterator` and `const_iterator` typedefs as exposition-only, but since LEWG didn't seem to object, R2 makes them non-exposition-only.

NOTE: We propose that `valarray`'s `.begin()` should be non-noexcept, for consistency with `.size()`. Adding `noexcept` consistently throughout `<valarray>` would be cool, but is out of scope.

Modify [template.valarray.overview] as follows:

```

namespace std {
    template<class T> class valarray {
    public:
        using value_type = T;
        using iterator = unspecified;
        using const_iterator = unspecified;

        // [valarray.cons], construct/destroy
        valarray();
        explicit valarray(size_t);

```

[...]

```

        // [valarray.range], range access

        iterator begin();
        iterator end();
        const_iterator begin() const;
        const_iterator end() const;

        // [valarray.members], member functions
        void swap(valarray&) noexcept;

        size_t size() const;

        T sum() const;
        T min() const;
        T max() const;

        valarray shift (int) const;
        valarray cshift(int) const;
        valarray apply(T func(T)) const;
        valarray apply(T func(const T&)) const;
        void resize(size_t sz, T c = T());
    };

```

§ 4.5. [valarray.members]

Move the existing section [\[valarray.range\]](#) from its current location to make it a sibling of [\[valarray.members\]](#); then modify it as follows:

28.6.10 28.6.2.x valarray range access [valarray.range]

1. In the `begin` and `end` function templates that follow, `unspecified1` is a type that The iterator type meets the requirements of a mutable `Cpp17RandomAccessIterator` ([random.access.iterators]) and models `contiguous_iterator` ([iterator.concept.contiguous]); whose its `value_type` is the template parameter `T` and whose its reference type is `T&`. `unspecified2` is a type that The const iterator type meets the requirements of a constant `Cpp17RandomAccessIterator` and models `contiguous_iterator`; whose its `value_type` is the template parameter `T` and whose its reference type is `const T&`.

2. The iterators returned by `begin` and `end` for an array are guaranteed to be valid until the member function `resize(size_t, T)` is called for that array or until the lifetime of that array ends, whichever happens first.

```
template<class T> unspecified1 begin(valarray<T>& v);  
template<class T> unspecified2 begin(const valarray<T>& v);  
iterator begin();  
const iterator begin() const;
```

3. Returns: An iterator referencing the first value in the array.

```
template<class T> unspecified1 end(valarray<T>& v);  
template<class T> unspecified2 end(const valarray<T>& v);  
iterator end();  
const iterator end() const;
```

4. Returns: An iterator referencing one past the last value in the array.

28.6.2.8 Member functions [valarray.members]

```
void swap(valarray& v) noexcept;
```

1. Effects: `*this` obtains the value of `v`. `v` obtains the value of `*this`.

2. Complexity: Constant.

§ 4.6. [support.initlist]

Modify [support.initlist] as follows:

[...]

17.10.2 Header `<initializer_list>` synopsis [initializer.list.syn]

```
namespace std {
    template<class E> class initializer_list {
    public:
        using value_type      = E;
        using reference       = const E&;
        using const_reference = const E&;
        using size_type       = size_t;

        using iterator        = const E*;
        using const_iterator  = const E*;

        constexpr initializer_list() noexcept;

        constexpr const E* data() const noexcept;
        constexpr size_t size() const noexcept;    // number of elements
        constexpr bool empty() const noexcept;
        constexpr const E* begin() const noexcept; // first element
        constexpr const E* end() const noexcept;   // one past the last element
    };

    // [support.initlist.range], initializer list range access
    template<class E> constexpr const E* begin(initializer_list<E> il) noexcept;
    template<class E> constexpr const E* end(initializer_list<E> il) noexcept;
}
```

1. An object of type `initializer_list<E>` provides access to an array of objects of type `const E`.

[Note: A pair of pointers or a pointer plus a length would be obvious representations for `initializer_list`. `initializer_list` is used to implement initializer lists as specified in [dcl.init.list]. Copying an `initializer_list` does not copy the underlying elements. — end note]

2. If an explicit specialization or partial specialization of `initializer_list` is declared, the program is ill-formed.

17.10.3 Initializer list constructors [support.initlist.cons]

```
constexpr initializer_list() noexcept;
```

1. Postconditions: `size() == 0`.

17.10.4 Initializer list access [support.initlist.access]

```
constexpr const E* begin() const noexcept;
```

1. Returns: A pointer to the beginning of the array. If `size() == 0` the values of `begin()` and `end()` are unspecified but they shall be identical.

```
constexpr const E* end() const noexcept;
```

2. Returns: `begin() + size()`.

```
constexpr const E* data() const noexcept;
```

x. Returns: `begin()`.

```
constexpr size_t size() const noexcept;
```

3. Returns: The number of elements in the array.

4. Complexity: Constant ~~time~~.

```
constexpr bool empty() const noexcept;
```

x. Returns: `size() == 0`.

~~17.10.5 Initializer list range access [support.initlist.range]~~

~~template<class E> constexpr const E* begin(initializer_list<E> il) noexcept;~~

~~1. Returns: `il.begin()`.~~

~~template<class E> constexpr const E* end(initializer_list<E> il) noexcept;~~

~~2. Returns: `il.end()`.~~

§ 4.7. [iterator.synopsis]

Modify [\[iterator.synopsis\]](#) as follows:

25.2 Header <iterator> synopsis [iterator.synopsis]

```
#include <compare>           // see [compare.syn]
#include <concepts>          // see [concepts.syn]
#include <initializer_list>  // see [initializer.list.syn]

namespace std {

[...

    // [iterator.range], range access
    template<class C> constexpr auto begin(C& c)
        noexcept(noexcept(c.begin())) -> decltype(c.begin());
    template<class C> constexpr auto begin(const C& c)
        noexcept(noexcept(c.begin())) -> decltype(c.begin());
    template<class C> constexpr auto end(C& c)
        noexcept(noexcept(c.end())) -> decltype(c.end());
    template<class C> constexpr auto end(const C& c)
        noexcept(noexcept(c.end())) -> decltype(c.end());
    template<class T, size_t N> constexpr T* begin(T (&array)[N]) noexcept;
    template<class T, size_t N> constexpr T* end(T (&array)[N]) noexcept;
    template<class C> constexpr auto cbegin(const C& c)
        noexcept(noexcept(std::begin(c))) -> decltype(std::begin(c));
    template<class C> constexpr auto cend(const C& c)
        noexcept(noexcept(std::end(c))) -> decltype(std::end(c));
    template<class C> constexpr auto rbegin(C& c)
        noexcept(noexcept(c.rbegin())) -> decltype(c.rbegin());
    template<class C> constexpr auto rbegin(const C& c)
        noexcept(noexcept(c.rbegin())) -> decltype(c.rbegin());
    template<class C> constexpr auto rend(C& c)
        noexcept(noexcept(c.rend())) -> decltype(c.rend());
    template<class C> constexpr auto rend(const C& c)
        noexcept(noexcept(c.rend())) -> decltype(c.rend());
    template<class T, size_t N> constexpr reverse_iterator<T*>
        rbegin(T (&array)[N]) noexcept;
    template<class T, size_t N> constexpr reverse_iterator<T*>
        rend(T (&array)[N]) noexcept;
    template<class E> constexpr reverse_iterator<const E*>
        rbegin(initializer_list<E> il) noexcept;
    template<class E> constexpr reverse_iterator<const E*>
        rend(initializer_list<E> il) noexcept;
    template<class C> constexpr auto crbegin(const C& c)
        noexcept(noexcept(std::rbegin(c))) -> decltype(std::rbegin(c));
    template<class C> constexpr auto crend(const C& c)
        noexcept(noexcept(std::rend(c))) -> decltype(std::rend(c));

    template<class C> constexpr auto size(const C& c)
        noexcept(noexcept(c.size())) -> decltype(c.size());
    template<class T, size_t N> constexpr size_t
        size(const T (&array)[N]) noexcept;

    template<class C> constexpr auto
        ssize(const C& c) noexcept(noexcept(c.size()))
        -> common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>;
    template<class T, ptrdiff_t N> constexpr ptrdiff_t
        ssize(const T (&array)[N]) noexcept;

    template<class C> constexpr auto empty(const C& c)
        noexcept(noexcept(c.empty())) -> decltype(c.empty());
    template<class T, size_t N> constexpr bool
        empty(const T (&array)[N]) noexcept;
    template<class E> constexpr bool
    empty(initializer_list<E> il) noexcept;

    template<class C> constexpr auto data(C& c)
```

```

    noexcept(noexcept(c.data())) -> decltype(c.data());
template<class C> constexpr auto data(const C& c)
    noexcept(noexcept(c.data())) -> decltype(c.data());
template<class T, size_t N> constexpr T* data(T (&array)[N]) noexcept;
template<class E> constexpr const E* data(initializer_list<E> il) noexcept;
}

```

§ 4.8. [iterator.range]

LWG approved this addition of `<stacktrace>` to resolve [\[LWG3625\]](#). LWG approved this addition of `<optional>` to resolve [\[LWG4131\]](#).

Modify [\[iterator.range\]](#) as follows:

1. In addition to being available via inclusion of the `<iterator>` header, the function templates in `[iterator.range]` are available when any of the following headers are included: `<array>`, `<deque>`, `<flat_map>`, `<flat_set>`, `<forward_list>`, `<inplace_vector>`, `<list>`, `<map>`, `<optional>`, `<regex>`, `<set>`, `<span>`, `<stacktrace>`, `<string>`, `<string_view>`, `<unordered_map>`, `<unordered_set>`, `<valarray>`, and `<vector>`.

```
template<class C> constexpr auto begin(C& c)
    noexcept(noexcept(c.begin())) -> decltype(c.begin());
template<class C> constexpr auto begin(const C& c)
    noexcept(noexcept(c.begin())) -> decltype(c.begin());
```

2. Returns: `c.begin()`.

```
template<class C> constexpr auto end(C& c)
    noexcept(noexcept(c.end())) -> decltype(c.end());
template<class C> constexpr auto end(const C& c)
    noexcept(noexcept(c.end())) -> decltype(c.end());
```

3. Returns: `c.end()`.

```
template<class T, size_t N> constexpr T* begin(T (&array)[N]) noexcept;
```

4. Returns: `array`.

```
template<class T, size_t N> constexpr T* end(T (&array)[N]) noexcept;
```

5. Returns: `array + N`.

```
template<class C> constexpr auto cbegin(const C& c) noexcept(noexcept(std::begin(c)))
    -> decltype(std::begin(c));
```

6. Returns: `std::begin(c)`.

```
template<class C> constexpr auto cend(const C& c) noexcept(noexcept(std::end(c)))
    -> decltype(std::end(c));
```

7. Returns: `std::end(c)`.

```
template<class C> constexpr auto rbegin(C& c)
    noexcept(noexcept(c.rbegin())) -> decltype(c.rbegin());
template<class C> constexpr auto rbegin(const C& c)
    noexcept(noexcept(c.rbegin())) -> decltype(c.rbegin());
```

8. Returns: `c.rbegin()`.

```
template<class C> constexpr auto rend(C& c)
    noexcept(noexcept(c.rend())) -> decltype(c.rend());
template<class C> constexpr auto rend(const C& c)
    noexcept(noexcept(c.rend())) -> decltype(c.rend());
```

9. Returns: `c.rend()`.

```
template<class T, size_t N> constexpr reverse_iterator<T*> rbegin(T (&array)[N]) noexcept;
```

10. Returns: `reverse_iterator<T*>(array + N)`.

```
template<class T, size_t N> constexpr reverse_iterator<T*> rend(T (&array)[N]) noexcept;
```

11. Returns: `reverse_iterator<T*>(array)`.

```
template<class E> constexpr reverse_iterator<const E*> rbegin(initializer_list<E> il) noexcept;
```


12. Returns: reverse_iterator<const E*>(il.end()).

```
template<class E> constexpr reverse_iterator<const E*> rend(initializer_list<E> il) noexcept;
```

13. Returns: reverse_iterator<const E*>(il.begin()).

```
template<class C> constexpr auto rbegin(const C& c)
    noexcept(noexcept(std::rbegin(c))) -> decltype(std::rbegin(c));
```

14. Returns: std::rbegin(c).

```
template<class C> constexpr auto rend(const C& c)
    noexcept(noexcept(std::rend(c))) -> decltype(std::rend(c));
```

15. Returns: std::rend(c).

```
template<class C> constexpr auto size(const C& c)
    noexcept(noexcept(c.size())) -> decltype(c.size());
```

16. Returns: c.size().

```
template<class T, size_t N> constexpr size_t size(const T (&array)[N]) noexcept;
```

17. Returns: N.

```
template<class C> constexpr auto ssize(const C& c)
    noexcept(noexcept(c.size())) -> common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>;
```

18. Effects: Equivalent to:

```
return static_cast<common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>>(c.size());
```

```
template<class T, ptrdiff_t N> constexpr ptrdiff_t ssize(const T (&array)[N]) noexcept;
```

19. Returns: N.

```
template<class C> constexpr auto empty(const C& c)
    noexcept(noexcept(c.empty())) -> decltype(c.empty());
```

20. Returns: c.empty().

```
template<class T, size_t N> constexpr bool empty(const T (&array)[N]) noexcept;
```

21. Returns: false.

```
template<class E> constexpr bool empty(initializer_list<E> il) noexcept;
```

~~22. Returns: il.size() == 0.~~

```
template<class C> constexpr auto data(C& c)
    noexcept(noexcept(c.data())) -> decltype(c.data());
template<class C> constexpr auto data(const C& c)
    noexcept(noexcept(c.data())) -> decltype(c.data());
```

23. Returns: c.data().

```
template<class T, size_t N> constexpr T* data(T (&array)[N]) noexcept;
```

24. Returns: array.

```
template<class E> constexpr const E* data(initializer_list<E> il) noexcept;
```

25. Returns: `il.begin()`.

§ 4.9. `.data+.size` cleanup

§ 4.9.1. `[string.cons]`

Modify `[string.cons]` as follows:

```
constexpr basic_string& operator=(initializer_list<charT> il);
```

36. *Effects*: Equivalent to:

```
return *this = basic_string_view<charT, traits>(il.begin(), il.data(), il.size());
```

§ 4.9.2. `[string.append]`

Modify `[string.append]` as follows:

```
constexpr basic_string& append(initializer_list<charT> il);
```

16. *Effects*: Equivalent to: return `append(il.begin(), il.data(), il.size());`

§ 4.9.3. `[string.assign]`

Modify `[string.assign]` as follows:

```
constexpr basic_string& assign(initializer_list<charT> il);
```

12. *Effects*: Equivalent to: return `assign(il.begin(), il.data(), il.size());`

§ 4.9.4. `[string.replace]`

Modify `[string.replace]` as follows:

```
constexpr basic_string& replace(const_iterator i1, const_iterator i2, initializer_list<charT> il);
```

12. *Effects*: Equivalent to: return `replace(i1, i2, il.begin(), il.data(), il.size());`

§ 4.9.5. `[span.cons]`

Modify `[span.cons]` as follows:

```
constexpr explicit(extent != dynamic_extent) span(std::initializer_list<value_type> il);
```

18. *Constraints*: `is_const_v<element_type>` is true.

19. *Preconditions*: If `extent` is not equal to `dynamic_extent`, then `il.size()` is equal to `extent`.

20. *Effects*: Initializes `data_` with `il.begin()` `il.data()` and `size_` with `il.size()`.

§ 4.9.6. [valarray.cons]

Modify [valarray.cons] as follows:

```
valarray(initializer_list<T> il);
```

9. *Effects*: Equivalent to `valarray(il.begin(), il.data(), il.size())`.

§ References

§ Informative References

[CWG2825]

Arthur O'Dwyer. *Range-based for statement using a braced-init-list*. November 2023. URL: <https://cplusplus.github.io/CWG/issues/2825.html>

[LWG2128]

Dmitry Polukhin. *Absence of global functions `cbegin/cend`*. January 2012–January 2016. URL: <https://cplusplus.github.io/LWG/issue2128>

[LWG2280]

Andy Sawyer. *begin/end for arrays should be constexpr and noexcept*. August 2013–January 2016. URL: <https://cplusplus.github.io/LWG/issue2280>

[LWG3537]

Jiang An. *Missing noexcept for `std::rbegin/rend` for arrays and `initializer_list`*. March 2021. URL: <https://cplusplus.github.io/LWG/issue3537>

[LWG3624]

Jiang An. *Inconsistency of `<typeinfo>`, `<initializer_list>`, and `<compare>`*. October 2021. URL: <https://cplusplus.github.io/LWG/issue3624>

[LWG3625]

Jiang An. *Should `<stacktrace>` provide range access function templates?*. October 2021. URL: <https://cplusplus.github.io/LWG/issue3625>

[LWG4001]

Hewill Kang. *iota_view should provide empty*. October 2023. URL: <https://cplusplus.github.io/LWG/issue4001>

[LWG4035]

Hewill Kang. *single_view should provide empty*. December 2023. URL: <https://cplusplus.github.io/LWG/issue4035>

[LWG4131]

Hewill Kang. *Including `<optional>` doesn't provide `std::begin/end`*. August 2024. URL: <https://cplusplus.github.io/LWG/issue4131>

[N3263]

J. Daniel Garcia. *More on noexcept for the Containers Library (revision)*. 25 March 2011. URL: <https://wg21.link/n3263>

[P2613]

Yihe Li. *Add the missing empty to mdspan*. June 2022. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2613r1.html>

[P3623R0]

Arthur O'Dwyer. *Add noexcept to [iterator.range] (LWG 3537)*. February 2025. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3623r0.html>